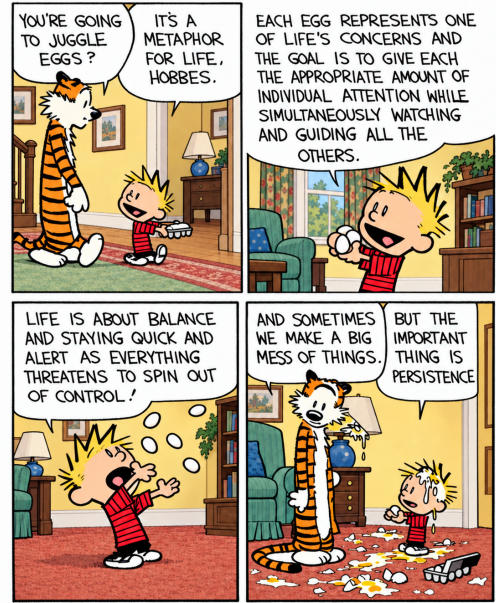


# CONTAINERIZATION & DOCKER

We will start by 9:05 p.m.



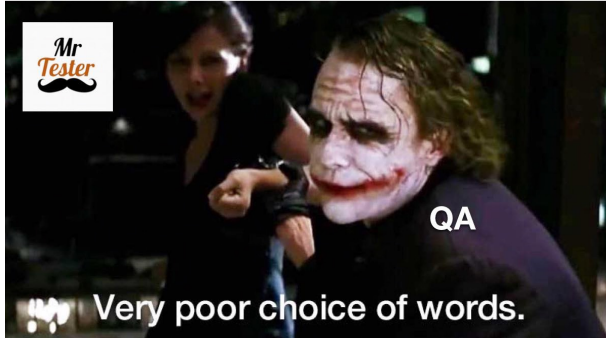
## AGENDA

- ① What is Docker??
- ② Diff b/w Docker, Virtual machine and virtual environments.
- ③ Workflow of Docker
- ④ Docker hub

# WHAT IS DOCKER

# WHY WE NEED IT?

Dev: Works in my machine...



Diff reasons why same codebase might break-down on client's system?

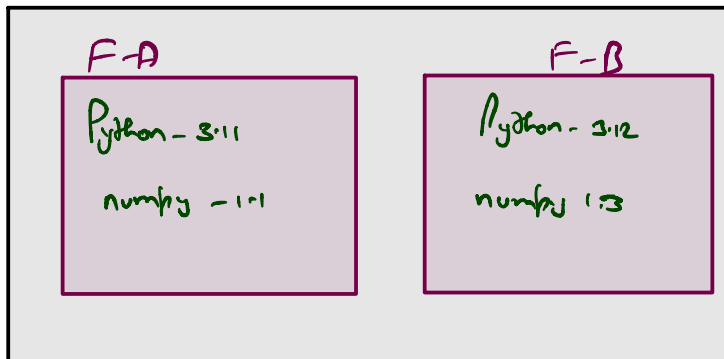
1. Library dependencies incompatibility.
2. Different Operating System.
3. Frontend, Backend, Database, etc...

Virtual Environments  
↳ Python



P-A - numpy 1.1

P-B numpy 1.3



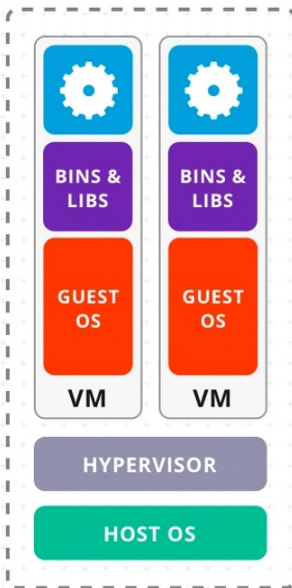
is conda similar to virtualenvwrapper?

How is Docker different than a virtual environment?

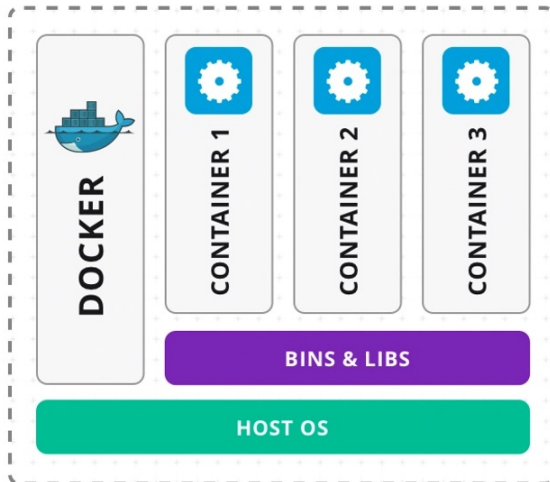
Docker → Isolated Environment.



Difference between a virtual machine and Docker container.



SERVER WITH  
VIRTUAL MACHINES



SERVER WITH  
DOCKER CONTAINERS

Today



Windows11\_Client\_x64\_en-us\_26300\_9457.iso



From <https://www.microsoft.com>

2.5 MB/s - 68.4 MB of 8.4 GB, 58 mins left

Intel or AMD 64-bit architecture

Download

5.9GB

|                      |                                        |
|----------------------|----------------------------------------|
| 64 GB RAM<br>2TB SSD | ① VM - windows<br>12 GB RAM, 50 GB SSD |
|                      | ① Kali Linux<br>12 GB RAM, 50 GB SSD   |

Pros of Virtual Machine:

1. You get to interact with the ui of a complete operating system.
2. You get to test on a, let's say, client's operating system.
3. Virtual machines are extremely secure, isolated.

In operating system, kernel is like the lowest level of software, and kernel interfaces directly with all the hardware and controls their application.

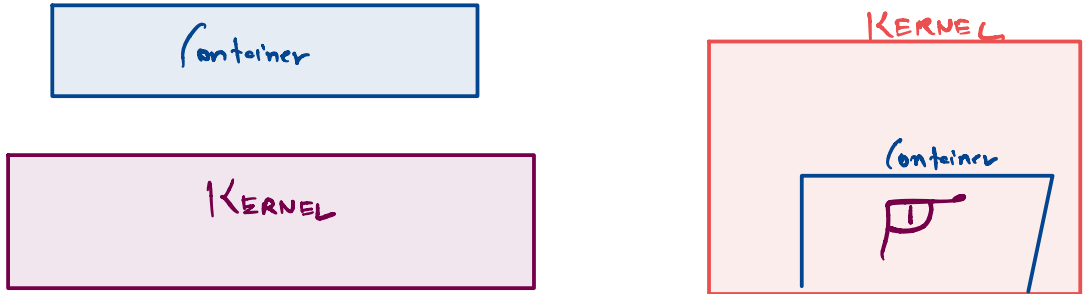
1. Process Management and Scheduling.
2. Allocation of RAM, memory management.
3. IO handling
4. Supervising security.



**alpine**  Docker Official Image · [↓ 1B+](#) · [☆ 10K+](#)

A minimal Docker image based on Alpine Linux with a complete package index and only 5 MB in size!

OPERATING SYSTEMS



### Steps for Deployment

1. Find some suitable cloud environment (AWS, GCP, Azure, etc).
2. Install or choose the operating system you want to work with, like Linux, Windows, etc
3. Install dependencies and libraries, including the front end, back end, database, etc.
4. Copy the code from the repository to the server.
5. And run the code and expose the URL/API.

### Docker Terminologies

1. DockerFile: Contains all the instructions of all the packages, codes, and any and all relevant files you're going to need for your application (Recipe).
2. Docker Image: We use Dockerfile to create Docker Images.
3. Docker Container: Deployed Docker Images, running instances of docker images.

# 1 Dockerfile = Recipe



The instructions for building and running our pasta.

# 2 Docker Image = Sealed Pasta Kit



A sealed kit with everything needed and startup instructions inside.

# 3 Docker Container = Cooked Serving



A running container: one cooked pasta serving, ready to enjoy.

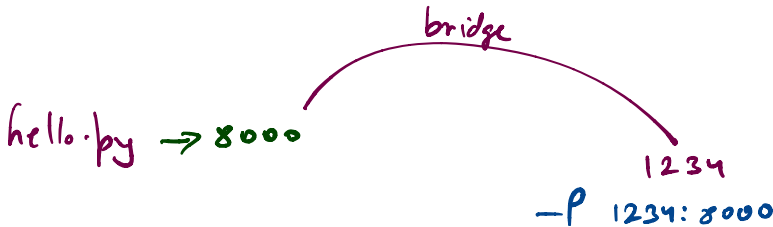
# 4 Docker = Kitchen System



The kitchen system that reads the recipe, builds the kit (image), and runs containers (servings) on demand.



localhost:1234



```
# Make port 8000 available to the world outside this container
CMD ["python", "-m", "flask", "--app", "hello.py", "run", "--host=0.0.0.0", "--port=8000"]
```

```
docker run -p 8000:9000 oct3
```

9000 bridge



P-1

launching Flask  
application

8000 available in  
container

P-2

-P 1234:8000



-P 7899:8888